

Code Migration and Software Environments

Yuwu Chen

HPC and Data Science Summer Institute

San Diego Supercomputer Center

August 3, 2026

<https://github.com/sdsc/sdsc-summer-institute-2025>

Outline

- Applications already available on Expanse (and other HPC systems)
 - environment modules
 - Singularity containers
- User built containers
 - Singularity build options
- Python based applications/libraries
 - SDSC installed and available via modules
 - Miniforge installs
 - uv, a fast modern Python package installer and resolver
- Installing/building applications from source code
 - Builds that utilize dependencies already installed on Expanse
 - Spack based approach for custom environments

Outline

- Applications already available on Expanse (and other HPC systems)
 - environment modules
 - Singularity containers
- User built containers
 - Singularity build options
- Python based applications/libraries
 - SDSC installed and available via modules
 - Miniforge installs
 - uv, a fast modern Python package installer and resolver
- Installing/building applications from source code
 - Builds that utilize dependencies already installed on Expanse
 - Spack based approach for custom environments

Applications already available on Expanse

- SDSC staff have installed and made available a large suite of libraries and applications on Expanse. This includes commercially licensed software such as MATLAB, Q-Chem, Abaqus, and Gaussian.
- The primary approach is to make applications available using environment modules.
- Some applications/frameworks (e.g. PyTorch, TensorFlow, AlphaFold) have been made available using Singularity containers.
- Today's talk will be focused on Expanse specific options. The options are broadly implementable on the Triton Shared Computing Cluster (TSCC), COSMOS, as the software stack on TSCC and COSMOS is installed w/ a similar approach as Expanse.
- On our Kubernetes based clusters (Voyager, Nautilus/PNRP) we are mainly using a containerized approach (docker images can be used). On Nautilus/PNRP we have a gitlab + Kubernetes development environment and a Coder (<https://coder.com/>) based development environment.

Why are environment modules used for loading applications?

- Environment variables
 - **PATH**: where to look for executables
 - **LD_LIBRARY_PATH**: where to look for shared libraries
 - **CPATH**: where to look for header and include files
- Other environment variables sometimes needed by various software
 - **LIBRARY_PATH**, **C_LIBRARY_PATH**, **LD_INCLUDE_PATH**
 - **LDFLAGS**, **LDLIBS**
- Environment Module
 - Application that **load/unload other applications on demand**.
 - Most ACCESS supercomputing sites use modules. Much more convenient than setting variables in ~/.bashrc
- Two types of Environment Modules
 - **Lmod**: a Lua-based environment module system (**Expansive**, TSCC and other SDSC HPC)
 - **Tcl**: an environment module in Tool Command Language (less common)

Modules: search application

- The command to find application is `module spider <application name>`.

```
[ychen64@login02 ~]$ module spider gcc
```

```
-----  
gcc:
```

```
-----  
Versions:
```

```
gcc/7.2.0
```

```
gcc/7.5.0
```

```
gcc/9.2.0
```

```
gcc/10.2.0
```

```
Other possible modules matches:
```

```
gcc-nvptx gcc/10.2.0 gcc/8.4.0 netcdf/gcc/64/gcc/64 openmpi/gcc/64
```

```
openmpi/mlnx/gcc/64  
-----
```

*NOTE:

`module spider` is only available in `Lmod` module. For `tcl` module, it is `module avail` to search application

Modules: search application

- The command to find application is `module spider <application name>`.

```
[ychen64@login02 ~]$ module spider gcc/10.2.0
```

```
-----  
gcc/10.2.0:
```

```
-----  
Versions:
```

```
    gcc/10.2.0/i62tgso
```

```
    gcc/10.2.0/npcyl14  
-----
```

For detailed information about a specific "gcc/10.2.0" package (including how to load the modules) use the module's full name. Note that names that have a trailing (E) are extensions provided by other modules.

For example:

```
$ module spider gcc/10.2.0/npcyl14  
-----
```

Modules: search application

- The command to find application is `module spider <application name>`.

```
[ychen64@login02 ~]$ module spider gcc/10.2.0/i62tgso
```

```
-----  
-----  
gcc/10.2.0: gcc/10.2.0/i62tgso  
-----  
-----
```

You will need to load all module(s) on any one of the lines below before the "gcc/10.2.0/i62tgso" module is available to load.

`gpu/0.17.3b`

Help:

The GNU Compiler Collection includes front ends for C, C++, Objective-C, Fortran, Ada, and Go, as well as libraries for these languages.

Modules: CPU and GPU software stacks on Expanse

- The module stacks are different for the CPU and GPU applications
- The paths can be enabled by loading the related CPU or GPU modules:
module load cpu #for CPU applications
module load gpu #for GPU applications
- Each CPU and GPU stack has two versions of 0.15.4 and 0.17.3b
- Do **not** mix the two as something compiled for the GPU stack will not work on the CPU nodes (which have a different architecture) and vice versa
- Application examples are provided in </cm/shared/examples/sdsc>

```
[ychen64@exp-1-02 ~]$ cd /cm/shared/examples/sdsc
```

```
[ychen64@exp-1-02 sdsc]$ ls
```

abacus	array	cp2k	gamess	hpcg	localscratch	namd	openacc	pyscf	raxml	tensorflow	vasp-ase
abinit	bintest	dftbplus	gaussian	hpl	matlab	neuron	openmp	pytorch	si	test	visit
alphafold	ciml	excerpt	gromacs	lammps	mpi	nsight	orca	qchem	siesta	trinity	wannier90
amber	classes	ExpanseAIR	hadoop	launcher	mpi-openmp-hybrid	nwchem	paraview	qe	spark	vasp	xpmem

Modules: load & unload application

- The command for loading a package into a user's environment is `module load <application name>`.
- If a specific version of an application is desired, the command can be expanded to `module load <application name>/<version>`.
- The command for unloading an application is `module unload <application name>`.

Modules: list, unload/reset all loaded applications

- The command to show what applications are currently loaded into a user's environment is `module list`
- To reset modules to default, use `module reset`

```
[ychen64@login02 ~]$ module reset
```

```
Resetting modules to system default. Resetting $MODULEPATH back to system default. All extra directories will be removed from $MODULEPATH.
```

```
[ychen64@login02 ~]$ module list
```

Currently Loaded Modules:

1) shared 2) cpu/0.17.3b (c) 3) slurm/expanse/23.02.7 4) sdsc/1.0 5) DefaultModules

Where:

c: built natively for AMD Rome

- To unload all loaded modules, use `module purge`

```
[ychen64@login02 ~]$ module purge
```

```
[ychen64@login02 ~]$ module list
```

```
No modules loaded
```

Modules: display the module changes

- The `module show` command will detail all changes that will be made to the user's environment: `module show <application name>`.

```
[ychen64@login02 ~]$ module show  cpu/0.17.3b
-----
/usr/share/modulefiles/cpu/0.17.3b.lua:
-----
whatis("Name      : cpu")
whatis("Version  : 0.17.3")
whatis("Release  : b")
whatis("Target   : zen")
whatis("Short Description : Adds modules for SDSC Software Stack for AMD / CPU Applications to MODULEPATH.")
help([[Adds modules for SDSC Software Stack for AMD / CPU Applications to MODULEPATH.
]])
add_property("arch","cpu")
unload("cpu","gpu","nostack")
setenv("SDSC_SPACK_STACK","cpu")
prepend_path("MODULEPATH","/cm/shared/apps/spack/0.17.3/cpu/b/share/spack/lmod/linux-rocky8-x86_64/Core")
```

Modules: auto-load modules

- Modules can be loaded automatically by adding the appropriate module load commands to a user's `~/ .bashrc`

```
# .bashrc
```

```
module load cpu/0.15.4 gcc/10.2.0 ncurses/6.2 intel-mkl/2019.1.144
```

Modules: create own module file

- Details about how to create own module file is placed in [Appendix](#)

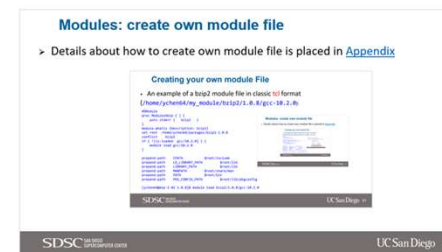
Creating your own module File

- An example of a bzip2 module file in classic **tcl** format (/home/ychen64/my_module/bzip2/1.0.8/gcc-10.2.0):

```
#%Module
proc ModulesHelp { } {
    puts stderr { bzip2 }
}
module-whatis {Description: bzip2}
set root /home/ychen64/packages/bzip2-1.0.8
conflict bzip2
if { ![is-loaded gcc/10.2.0] } {
    module load gcc/10.2.0
}

prepend-path CPATH $root/include
prepend-path LD_LIBRARY_PATH $root/lib
prepend-path LIBRARY_PATH $root/lib
prepend-path MANPATH $root/share/man
prepend-path PATH $root/bin
prepend-path PKG_CONFIG_PATH $root/lib/pkgconfig

[ychen64@exp-1-02 1.0.8]$ module load bzip2/1.0.8/gcc-10.2.0
```



Short summary: useful module commands

Module commands	Description
<code>module spider</code>	Search a certain application module
<code>module load / module unload</code>	Load / unload module
<code>module list</code>	List currently loaded modules
<code>module reset</code>	Reset modules to the default
<code>module purge</code>	Unload all loaded modules
<code>module show</code>	Detail all changes that will be made to the user's environment when module is loaded

Hands-on session 1: working with modules

https://github.com/sdsc/sdsc-summer-institute-2026/tree/main/2.4_code_migration_and_software_environments

- Run module commands to get the system-wide installed MATLAB software
 1. Search with the module string to show how to load a particular MATLAB
 - 1) Search matlab with `module spider`, and look for the name of version 2022b
 - 2) Search matlab version 2022b with `module spider`
Q1: How many flavors does matlab 2022b have?
 - 3) Search the full name of matlab 2022b with `module spider`
Q2: Which matlab 2022b module is built on the **CPU** software stack?
 2. Load MATLAB 2022b from the **CPU** software stack
 - 1) Reset the modules to default
 - 2) List current loaded module
 - 3) Load GPU stack and matlab 2022b
 - 4) List current loaded module
 - 5) Get what has changed by loading the MATLAB module
 3. Check if the path to the MATLAB executable is correctly set by `which matlab`
 - 1) Run `which matlab`

Applications available via Singularity Containers

- Some applications are easier to make available via Singularity containers.
- On Expanse the containers are at:
 - /cm/shared/apps/containers/singularity
 - /expanse/projects/qstore/installs/containers/singularity
- Applications available via Singularity include:
 - TensorFlow, PyTorch, AlphaFold, Paraview, VisIt
 - We also have the Extreme-scale Scientific Software Stack (E4S) available via a container. Ref: <https://e4s-project.github.io>
- The Singularity definition files are available* for users who wish to add to the containers/rebuild them.

Why is Singularity widely used on HPC?

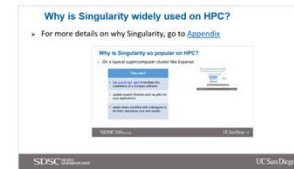
- For more details on why Singularity, go to [Appendix](#)

Why is Singularity so popular on HPC?

- On a typical supercomputer cluster like Expanse

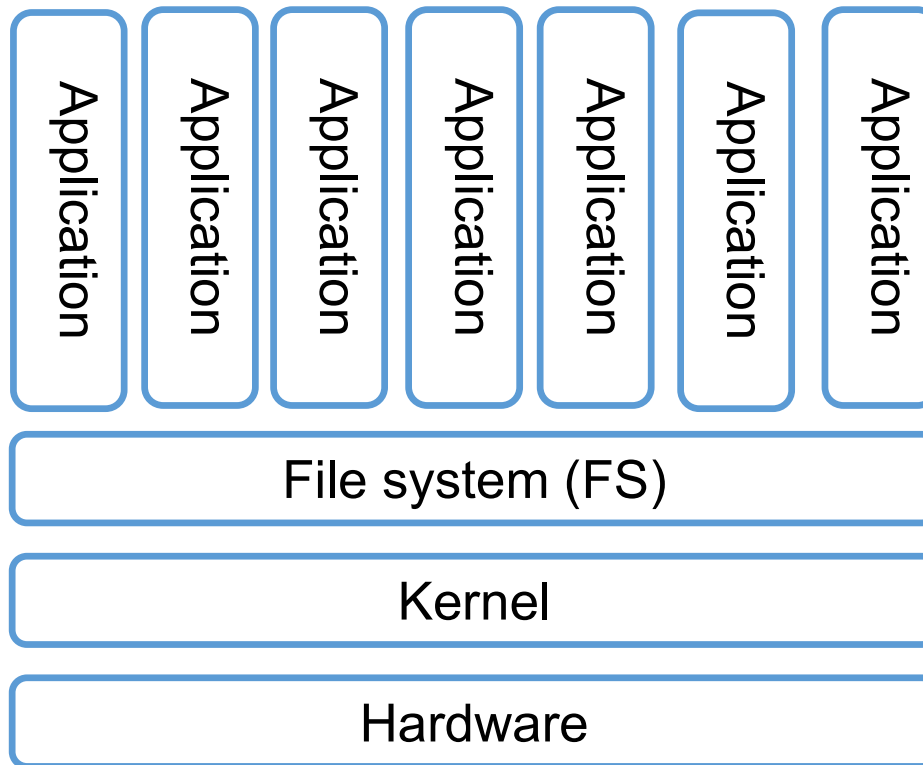
You can't

- run `yum` or `apt-get` to facilitate the installation of a complex software
- update system libraries such as `glibc` for your applications
- easily share workflow with colleagues to let them reproduce your test results

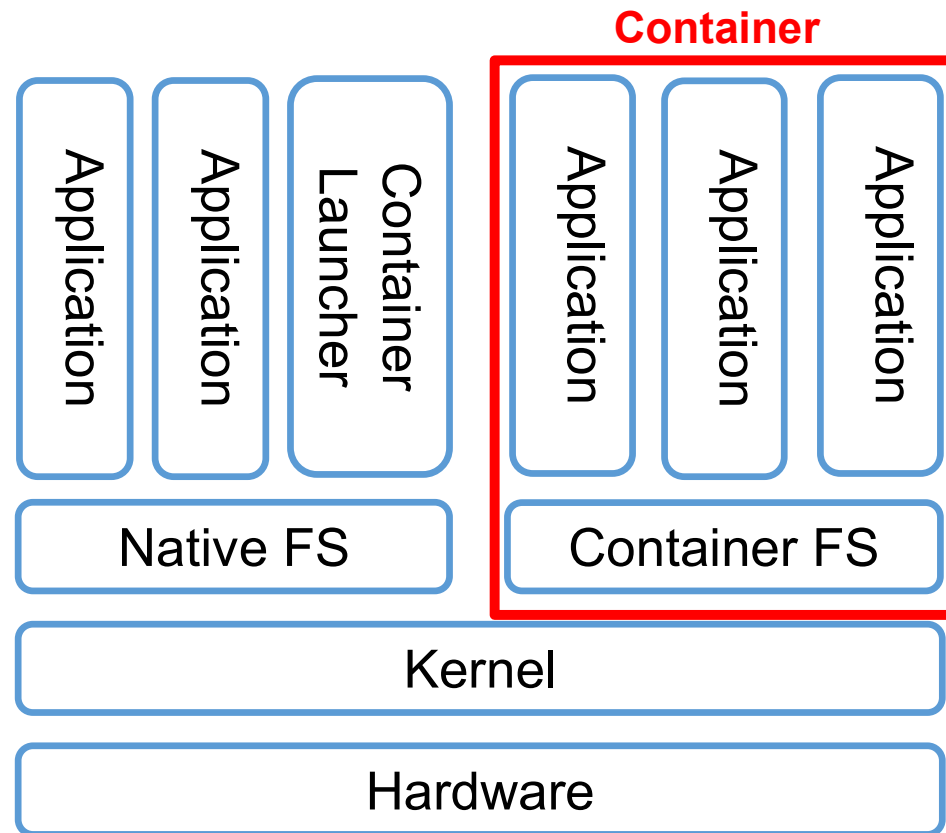


What is Singularity?

- Review Linux system architecture

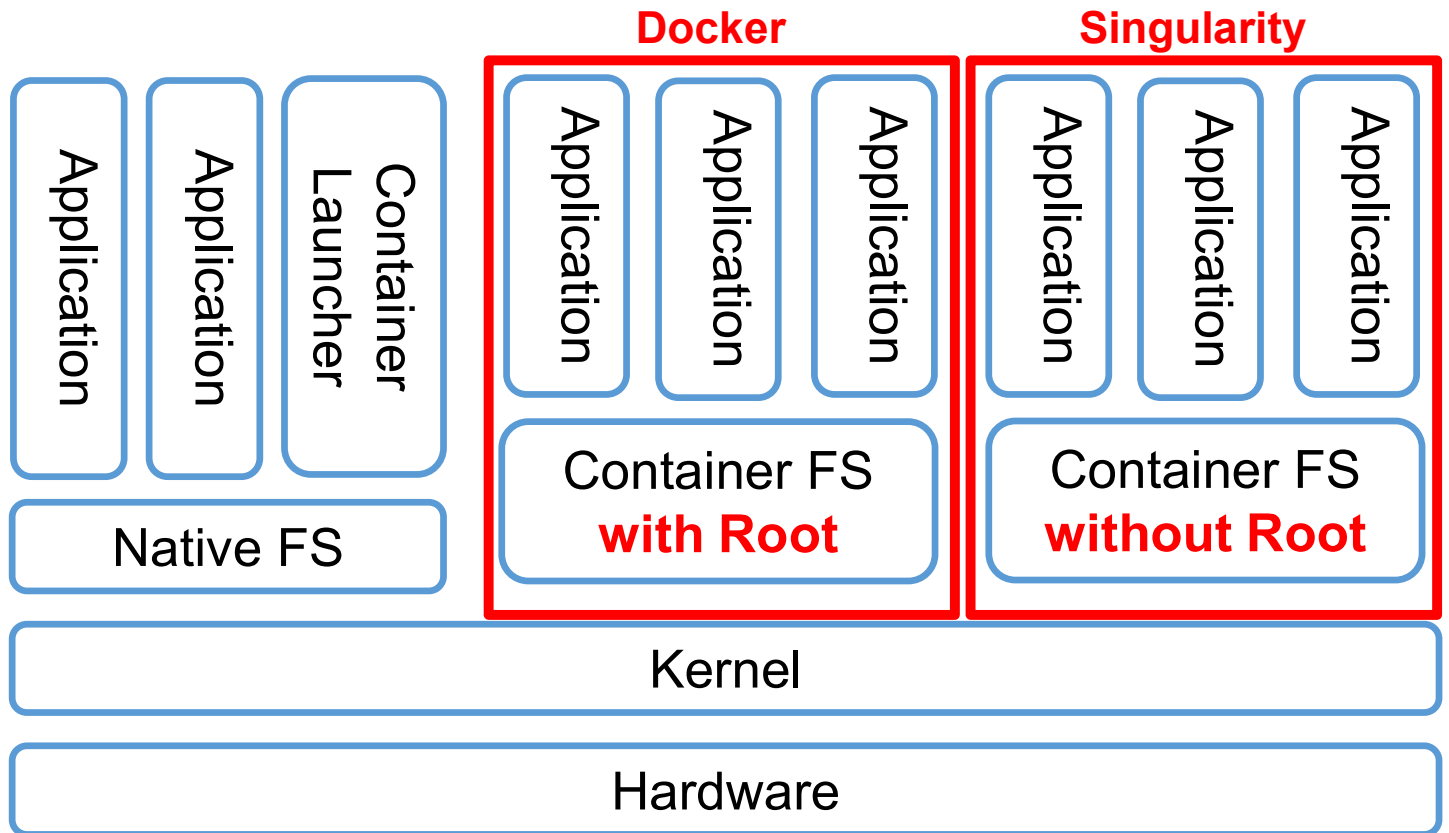


Container is a technology for packaging up applications



Singularity is a container technology for HPC

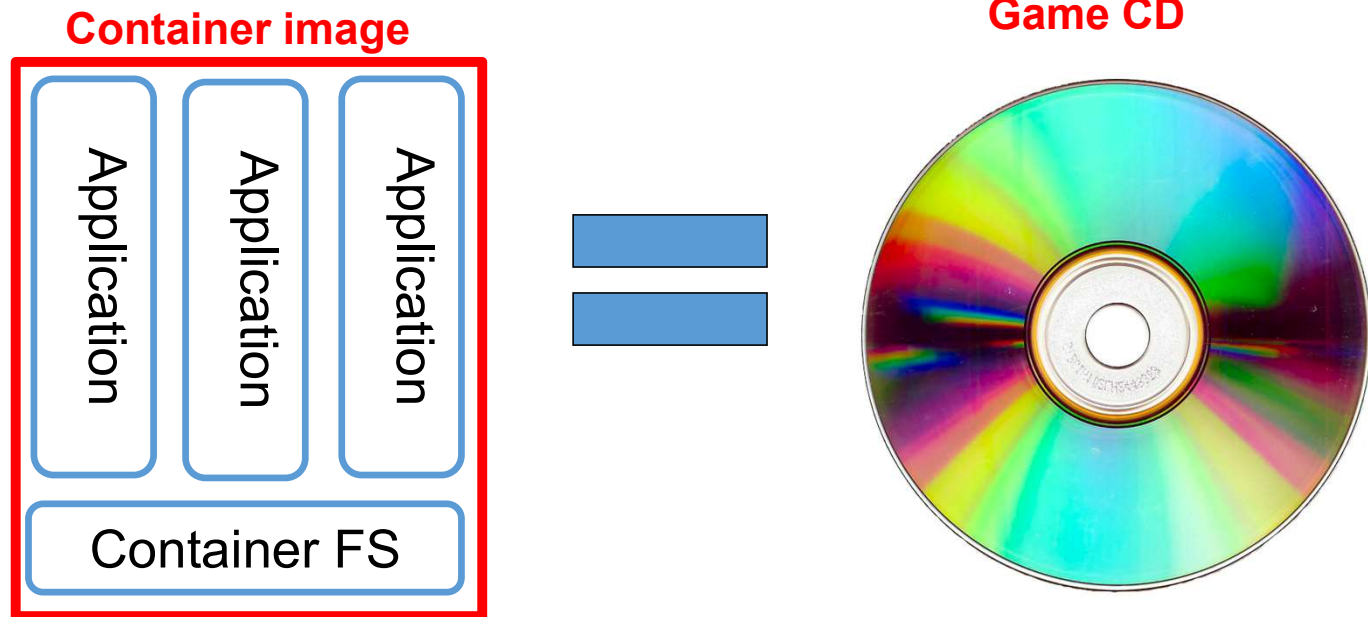
For security reason, root is not allowed when running container in the HPC environment



Container image is a static file that includes executable code

➤ Container image features:

- immutable
- portable



Applications available via Singularity Containers

- Some applications are easier to make available via Singularity containers.
- On Expanse the containers are at:
 - /cm/shared/apps/containers/singularity
 - /expanse/projects/qstore/installs/containers/singularity
- Applications available via Singularity include: **Many** 
 - TensorFlow, PyTorch, AlphaFold, Paraview, VisIt
 - We also have the Extreme-scale Scientific Software Stack (E4S) available via a container. Ref: <https://e4s-project.github.io>

Using Applications via Singularity Containers

```
#!/usr/bin/env bash
#SBATCH --job-name=pytorch-cpu-shared
#SBATCH --account=XYZ123
#SBATCH --partition=shared
#SBATCH --nodes=1
#SBATCH --ntasks-per-node=10
#SBATCH --cpus-per-task=1
#SBATCH --mem=90G
#SBATCH --time=00:30:00
#SBATCH --output=pytorch-cpu-shared.o%j.%N
```

module reset

module load singularitypro

```
time -p singularity exec --bind /expance,/scratch --nv /cm/shared/apps/containers/singularity/pytorch/pytorch-
latest.sif python3 $SLURM_SUBMIT_DIR/main.py
```


Outline

- Applications already available on Expanse (and other HPC systems)
 - environment modules
 - Singularity containers
- **User built containers**
 - **Singularity build options**
- Python based applications/libraries
 - SDSC installed and available via modules
 - Miniforge installs
 - uv, a fast modern Python package installer and resolver
- Installing/building applications from source code
 - Builds that utilize dependencies already installed on Expanse
 - Spack based approach for custom environments

Building Singularity containers

- Useful if there are a lot of dependencies that cannot be easily installed in the regular Expanse environment
- On Expanse, run `singularity build` to create simple singularity images **from other images**
 - Example: building a container of fortune + cowsay

```
[ychen64@exp-9-55 ~]$ singularity build lolcow.sif docker://godlovedc/lolcow
```

- `lolcow.sif` is the singularity image name with an extension of `.sif`
- `docker://` is an external resource where Singularity download the pre-built image.
 - `docker://` refers to Docker Hub container image uri

Building Singularity containers

- For MPI based installs:
 - The MPI in the container should match the external MPI version
 - Make sure the InfiniBand drivers are installed in the container
 - We have example definition files available
- For GPU installs:
 - Make sure packages installed are compatible with driver on our system
 - Nvidia GPU Cloud (NGC) <https://www.nvidia.com/en-us/gpu-cloud/>
 - NGC examples

[docker://nvcr.io/nvidia/tensorflow:20.09-tf2-py3](https://nvcr.io/nvidia/tensorflow:20.09-tf2-py3)

Building Singularity containers

- The Singularity definition files are available* for users who wish to add to the containers/rebuild them on Expanse.
 - * <https://github.com/mkandes/naked-singularity/tree/master/definition-files>
- Builds from definition files need root access and cannot be done on Expanse.
 - Build elsewhere - for example use your laptop/desktop OR a cloud resource (e.g. Jetstream2 on ACCESS)

Sample Singularity definition file

```
Bootstrap: shub
From: mkandes/naked-singularity:centos-7.9.2009

%labels

    APPLICATION_NAME centos + mvapich
    APPLICATION_VERSION 7.9.2009 + 2.3.2
    APPLICATION_URL https://mvapich.cse.ohio-state.edu

    AUTHOR_NAME Marty Kandes
    AUTHOR_EMAIL mkandes@sdsc.edu

    LAST_UPDATED 20201227

%setup

%environment

    # Set paths to MVAPICH2 binaries and libraries
    export PATH="/opt/mvapich2-2.3.2/bin:${PATH}"
    export LD_LIBRARY_PATH="/opt/mvapich2-2.3.2/lib:${LD_LIBRARY_PATH}"

%post -c /bin/bash

    # Set operating system mirror URL
    export MIRRORURL='http://mirror.centos.org/centos-7/7.9.2009/os/x86_64'

    # Set operating system version
    export OSVERSION='7'

    # Set system locale
    export LC_ALL=C

    # Update all software packages to their latest versions
    yum -y check-update && yum -y update

    # Install basic drivers for user space access to Ethernet, RDMA,
    # and Infiniband. See https://community.mellanox.com/docs/DOC-2431
    yum -y install dkms
    yum -y install infiniband-diags
    yum -y install infiniband-diags-devel
    yum -y install libibverbs
```

```
yum -y install libibverbs-devel
yum -y install ibacm
yum -y install librdmacm
yum -y install librdmacm-devel
yum -y install libmlx4
yum -y install libmlx5
yum -y install mstflint
yum -y install libibcm
yum -y install libibmad
yum -y install libibmad-devel
yum -y install libibumad
yum -y install libibumad-devel
yum -y install opensm
yum -y install srptools

# Install additional tools
yum -y install ibutils
yum -y install libibverbs-utils
yum -y install librdmacm-utils
yum -y install perftest
yum -y install numactl

# Install libnl
yum -y install libnl3
yum -y install libnl3-devel

# Install mvapich2 (build) dependencies
yum -y install bison

cd /tmp

# Download, build, and install mvapich2
wget http://mvapich.cse.ohio-state.edu/download/mvapich/mv2/mvapich2-2.3.2.tar.gz
tar -xzf mvapich2-2.3.2.tar.gz
cd mvapich2-2.3.2
./configure --prefix=/opt/mvapich2-2.3.2
make
make install

# Cleanup
package-cleanup -q --leaves | xargs -l1 yum -y remove
yum -y clean all
```

More Singularity training at SDSC

- Expanse Webinar: Singularity – Containers for Scientific and High-Performance Computing

<https://education.sdsc.edu/training/interactive/?id=202203> [ExpanseWebinar-M.Kandes](#)

- SDSC Webinar: Introduction to Singularity

<https://education.sdsc.edu/training/interactive/202210> [TSCCwebinar-Singularity?id=202305-SDSCWebinar-Singularity-Containers](#)

Short summary: Singularity containers on Expanse

- Many applications are available via Singularity containers
 - /cm/shared/apps/containers/singularity
 - /expanse/projects/qstore/installs/containers/singularity
 - TensorFlow, PyTorch, AlphaFold, Paraview, VisIt, Extreme-scale Scientific Software Stack (E4S)
 - Singularity definition files are available
- Building Singularity containers
 - `singularity build` to create singularity images from other images (can build on Expanse)
 - Building from a Singularity definition file. (your laptop/desktop OR a cloud resource)
 - To run MPI or GPU on Expanse, must choose compatible MPI or GPU packages

Hands-on session 2: working with Singularity containers

https://github.com/sdsc/sdsc-summer-institute-2026/tree/main/2.4_code_migration_and_software_environments

- Use a PyTorch Singularity image for the software environment.
 1. Copy over a PyTorch example from the Expanse examples directory
`/cm/shared/examples/sdsc/pytorch`
 2. Submit job with `sbatch`. Remember to update your allocation name
 3. Check the output file. Where is the data saved at job runtime? Where is the Slurm output file saved?
- Build a Singularity image on Expanse
 1. Start an interactive session
 2. Load Singularity module (module name is `singularitypro`)
 3. Build an image `lolcow.sif` from `docker://godlovedc/lolcow`
 4. Run this image with `singularity exec lolcow.sif <commands>`
 - 1) command1: `fortune`
 - 2) command2: `cowsay <text>`
 - 3) command3: `sh -c 'fortune | cowsay'`

Outline

- Applications already available on Expanse (and other HPC systems)
 - environment modules
 - Singularity containers
- User built containers
 - Singularity build options
- Python based applications/libraries
 - SDSC installed and available via modules
 - Miniforge installs
 - uv, a fast modern Python package installer and resolver
- Installing/building applications from source code
 - Builds that utilize dependencies already installed on Expanse
 - Spack based approach for custom environments

Python applications/libraries via modules

- Several python applications and libraries are available via modules
 - Examples
py-matplotlib, py-numpy, py-scipy
py-pysam, py-phonopy, py-htseq
- Use `module spider py-` and `module show` to get more information
- Some of these modules are dependent on MPI like py-pyscf and py-mpi4py
- Use modules-based approach if all the packages needed are in modules **but pay attention to the package version available via modules.**
- If further installs are needed, it's best to go to the conda (Miniforge) or Singularity approach.
- Do NOT mix the python apps/libraries from the modules with ones from conda (Miniforge) or a container. They will likely conflict/fail.

mpi4py

- mpi4py install needs to be consistent with the MPI being used. The system installed versions are:

CPU/GPU module	gcc module	mpi implementation module	mpi4py module
cpu/0.17.3b	gcc/10.2.0/npcyll4	intel-mpi/2019.10.317/kdx4qap	py-mpi4py/3.1.2/cllp7nt
cpu/0.17.3b	gcc/10.2.0/npcyll4	openmpi/4.1.3/oq3qvsv	py-mpi4py/3.1.2/kas2whp
cpu/0.17.3b	gcc/10.2.0/npcyll4	mvapich2/2.3.7/iyjtn3x	py-mpi4py/3.1.2/silsqln
gpu/0.17.3b	gcc/10.2.0/i62tgso	intel-mpi/2019.10.317/jhyxn2g	py-mpi4py/3.1.2/3lfjrdm
gpu/0.17.3b	gcc/10.2.0/i62tgso	openmpi/4.1.3/gzzscfu	py-mpi4py/3.1.2/7ebfcgr

- Note that if you do a conda install of mpi4py, it will use a conda based mpi install (and not the system one). This is ok for single node cases, but multi-node will not use the high-performance InfiniBand network.
- Use system installed MPI and combine with the conda install by building mpi4py from source.

Installs using Miniforge conda installer

- Use Miniforge* conda (mamba) installs if the software available on the system doesn't cover your needs. Specially if custom installs with both python and non-python dependencies are required.
- Miniforge3 is replacing Anaconda3 and Miniconda3
 - Includes `mamba` by default
 - `mamba install` is faster and easier to troubleshoot than `conda install`

* <https://github.com/conda-forge/miniforge>

Installing Miniforge3

1. Start an interactive session on a compute or GPU node.
2. Download the installer script with `wget`
3. Install Miniforge3
 - can choose the default installation location `~/miniforge3`
 - If installing Miniconda3/Anaconda3, at the end of the installation, don't choose adding the installed paths to your `~/.bashrc`.
4. Create an alias to load it when needed

```
alias miniforge='eval "$($HOME/miniforge3/bin/conda shell.bash hook)''
```
5. Optional step: evaluate mamba's own hook to run `mamba activate`

```
eval "$($HOME/miniforge3/bin/mamba shell hook --shell bash)"
```

Conda/mamba command cheat sheet

conda/mamba	Description
<code>mamba info</code>	show current mamba environment info
<code>mamba list</code>	list package details in the current environment by line
<code>mamba env list</code>	List all available environments
<code>mamba search --channel <channel name> <lib name></code>	search a certain python lib in a certain channel. Default channel is conda-forge for mamba
<code>mamba create -n <env name></code>	create a conda environment
<code>mamba update/remove <lib name></code>	update/remove an installed python lib
<code>conda activate/deactivate <env name></code>	activate/deactivate a conda environment

A few notices about conda (mamba) install

- Keep your **base** environment small
 - Generally, let only conda and mamba happily live in the base environment
- Install conda environment in your home directory.

Do NOT run conda (mamba) installs into any Lustre location - can cause systemwide problems due to metadata loads.
- For GPU installs, make sure the packages chosen work with driver on Expanse. For example, the current versions are:
 - NVIDIA-SMI 525.85.12 Driver Version: 525.85.12 CUDA Version: 12.0
 - Make sure any conda based installs are with a CUDA version that is 12.0 or older.

Use uv to replace pip and virtualenv

- uv is a single tool to replace pip, pip-tools, pipx, poetry, pyenv, twine, virtualenv, and more.
- 10-100x faster* than pip
- With a universal lockfile, it can manage different projects in various Python versions.
- NOT a replacement of conda environment
- Install uv with `curl` or `wget`:

```
# curl -LsSf https://astral.sh/uv/install.sh | sh
# wget -qO- https://astral.sh/uv/install.sh | sh
```

* <https://github.com/astral-sh/uv/blob/main/BENCHMARKS.md>

Getting started with uv

- Running uv as pip:

```
uv pip install <package name>
```

Ideal replacement but only used a small part of uv!

Getting started with uv

- uv project:

- # Create a new project

- uv init myproj

- cd myproj

- # Create environment + install dependencies

- uv add numpy pandas

- # Run Python inside the project environment

- uv run python

- Each project gets its own isolated environment (.venv)

Project structure in uv

- uv can have project-local environments instead of large shared conda environments.

```
(base) [ychen64@login2 myproj]$ ls -a
.  ..  .git  .gitignore  main.py  pyproject.toml  .python-version  README.md
uv.lock  .venv
```

Managing Python versions

- uv can have project-local environments instead of large shared conda environments.

```
# Install Python versions
uv python install 3.10 3.11 3.12
# Create project using Python 3.11
uv init myproj
cd myproj
uv python pin 3.11
uv sync
```

uv vs pip vs conda/mamba

Tool	Main Purpose
<code>pip</code>	Install Python packages
<code>uv</code>	Fast project/package/environment management
<code>conda/mamba</code>	Manage Python + MPI/CUDA/system libraries

- `uv` can replace `pip` but NOT `conda/mamba`

R based applications

- For more details on R based applications, go to [Appendix](#)

R applications/packages via modules

- Several R packages are available via modules
- Use `module spider r-` to find all installed apps/libraries.
- Examples
 - r-biobase, r-deseq2, r-doparallel, r-ggplot2
- Use modules-based approach if all the packages needed are in modules **but pay attention to the package version available via modules.**
- If further installs are needed, it is best to install from source, Miniforge3 conda/mamba install or Singularity approach.
- Don't mix the R apps/libraries from the modules with ones from source, conda installs or a container. They will likely conflict/fail.



Short summary: Python and R based applications

- Python packages are available
 - via modules. Check their versions before use
 - via conda/mamba install. Conda/mamba can be from Miniforge3
 - via Singularity container
- R based applications and packages are available
 - via modules. Check their versions before use
 - via installing from source (beware of getting incompatible binaries)
 - via conda/mamba install
 - via Singularity container

Hands-on session 3: working with Python or R (pick one)

https://github.com/sdsc/sdsc-summer-institute-2026/tree/main/2.4_code_migration_and_software_environments

- The Python part includes installing miniforge3 and pyjokes
 1. Download and install miniforge3, make it available by alias
 2. Create and activate a conda environment
 3. Install pyjokes and run it
- The R part includes load/build R and R package `praise`. Choose either one of below to add R to your user environment:
 1. Load the system R module and install it with `install.packages()`
 - 1) run module commands to search and load R
 - 2) start an R console with command `R`
 - 3) install praise from source with `install.packages()` and run with `praise()`
 2. Create an R conda environment with miniforge3
 - 1) Download and install miniforge3, make it available by alias
 - 2) create an R conda environment
 - 3) install praise and run

Outline

- Applications already available on Expanse (and other HPC systems)
 - environment modules
 - singularity containers
- User built containers
 - Singularity build options
- Python based applications/libraries
 - SDSC installed and available via modules
 - Miniforge installs
 - uv, a fast modern Python package installer and resolver
- **Installing/building applications from source code**
 - **Builds that utilize dependencies already installed on Expanse**
 - **Spack based approach for custom environments**

Installs from source, configure example

- Some applications have configure scripts that pick up dependencies based on environment variables and options.
- Example: Quantum Espresso
- Build using a job script:
 - /cm/shared/examples/sdsc/qe/cpu_stack_0.15.4/build_scripts/qe_gcc92_openmpi.sh
- Compile environment:

```
module reset
module load cpu/0.15.4
module load gcc/9.2.0
module load openmpi/3.1.6
module load amdblis
module load amdlibflame
module load amdfftw
```

Installs from source, QE example continued

- Compiler flags

CC=gcc

CXX=g++

F77=gfortran

FC=gfortran

F90=gfortran

export CFLAGS="-O3 -march=core-avx2"

export CXXFLAGS="-O3 -march=core-avx2"

export FCFLAGS="-O3 -march=core-avx2 "

export F90FLAGS="-O3 -march=core-avx2 -cpp "

export F77FLAGS="-O3 -march=core-avx2 "

export FFLAGS="-O3 -march=core-avx2 "

Installs from source, QE example continued

- Environment variables

```
export FFT_LIBS="-L${AMDFFTWHOME}/lib -lfftw3"
export FFT_INCLUDE="-I${AMDFFTWHOME}/include"
export FFTW_INCLUDE="-I${AMDFFTWHOME}/include"
export BLAS_LIBS="-L${AMDBLISHOME}lib -lblis"
export LAPACK_LIBS="-L${AMDLIBFLAMEHOME}/lib -lflame"
export IFLAGS="-I../include -I${AMDFFTWHOME}/include -I${AMDBLISHOME}/include -I${AMDLIBFLAMEHOME}/include"
export SCALAPACK_LIBS="-L${HOME}/scalapack/lib -lscalapack"
```

Installs from source, QE example continued

```
./configure \  
  CC=$CC \  
  CXX=$CXX \  
  F77=$F77 \  
  FC=$FC \  
  F90=$FC \  
  --prefix=$HOME/qe
```

```
cp make.sys make.sys.bak  
make $ESPRESSO_TARGETS  
make install
```

Installs from source, RAxML makefile example

```
F90 = gfortran
# FFLAGS = -Wall -fbounds-check
# FFLAGS = -g -Wall -fcheck=all
FFLAGS = -O2
LIBFLAGS = -L/cm/shared/apps/spack/cpu/opt/spack/linux-centos8-zen2/gcc-10.2.0/openblas-0.3.10-3lzjcwjsyu3qmott7k3k52mtzljoax3/lib -L/cm/shared/apps/spack/cpu/opt/spack/linux-centos8-zen2/gcc-10.2.0/fftw-3.3.8-zwmcmd2v5albxc2n5u5njzbou6r5vn3u/lib -lopenblas -lfftw3
INCFLAGS = -I/cm/shared/apps/spack/cpu/opt/spack/linux-centos8-zen2/gcc-10.2.0/openblas-0.3.10-3lzjcwjsyu3qmott7k3k52mtzljoax3/include -I/cm/shared/apps/spack/cpu/opt/spack/linux-centos8-zen2/gcc-10.2.0/fftw-3.3.8-zwmcmd2v5albxc2n5u5njzbou6r5vn3u/include
PDFL = pdflatex -synctex=1

DRIVERS = driver1 driver2 driver3 driver4

default: oz.so

all : $(DRIVERS) fftw_test oz.so

docs : oz_doc.pdf

oz_doc.pdf : oz_doc.tex gofr.png sof.k.png
    $(PDFL) oz_doc.tex
    $(PDFL) oz_doc.tex
    $(PDFL) oz_doc.tex

drivers : $(DRIVERS)

oz.so : oz_mod.f90
    f2py3 --overwrite-signature $< -m oz -h oz.pyf
    f2py3 -c $< oz.pyf $(INCFLAGS) $(LIBFLAGS)
```

Installs from source

- All GPU compiles **must** be done on a GPU node. Also, don't mix installs from GPU and CPU stack
- Compiles done on the login node will fail on a GPU node
- CPU codes can be compiled on the login node as the processor matches
- Note on BLAS/LAPACK/SCALAPACK: there are several options
 - OpenBLAS,
 - Netlib Scalapack
 - MKL
 - AOCL
- Libraries will be in non-standard locations. So, make sure to use configure/cmake/makefile options to point build scripts to the right locations
- Intel compilers work fine on AMD nodes. Don't use "-xHOST", switch to "-march=core-avx2"

Installs from source – Spack based installs

- Spack is a package manager that makes it easy to install scientific software on HPC systems

<https://spack.io/>

- SDSC maintains scripts and spack package repositories for machine specific installs

<https://github.com/sdsc/spack>

- AMD provides performant Spack recipes and specs:

<https://www.amd.com/en/developer/zen-software-studio/applications/spack.html>

```
module reset
git clone --depth=100 --branch=releases/v0.22 https://github.com/spack/spack.git ~/spack
. $HOME/spack/share/spack/setup-env.sh
spack compiler find
spack env create vasp-aocc
spack env activate vasp-aocc
spack install --add aocc@4.2.0 +license-agreed
spack compiler add
spack install --add openmpi@4.1.6 % aocc@4.2.0 ~atomics~cuda~cxx~cxx_exceptions~gpfs~internal-
hwloc~java+legacylaunchers+lustre~memchecker+pmi+romio~rsh~singularity+static+vt+wrapper-rpath fabrics=ucx,cma
schedulers=slurm ^ucx@1.14.0 ^lustre@2.15.4 ^slurm@23.02.7 ^rdma-core@58
spack install vasp +scalapack +openmp +fftw %aocc ^amdfw ^amdblis threads=openmp ^amdlibflame ^amdscalapack ^openmpi
```


Hands-on session 4: installing from source

https://github.com/sdsc/sdsc-summer-institute-2026/tree/main/2.4_code_migration_and_software_environments

- Start an interactive session, if the interactive session started in Hands-on sessions 2 or 3 has ended
- This example compiles cmatrix from source on Expanse
 1. Set up module environment. Make sure these three modules are loaded:
1) `cpu/0.15.4` 2) `gcc/10.2.0` 3) `ncurses/6.2`
 2. Download and build cmatrix from source, add the bin path to PATH
 - 1) Download from cmatrix repo <https://github.com/abishekvashok/cmatrix> with `git clone`
 - 2) `cd` to `cmatrix` directory
 - 3) Update generated configuration files with `autoreconf -i`
 - 4) Configure to specify the install path and wide-character ncurses
`./configure --prefix=$HOME/cmatrix/install LIBS="-lncursesw"`
 - 5) compile with `make` and install with `make install`
 3. Add the bin path to PATH
 4. Run executable `cmatrix`

Take home message

- Applications already available on Expanse (and other systems)
 - environment modules
 - Singularity containers
 - User built containers
 - Python based applications/libraries
 - SDSC installed and available via modules
 - Miniforge installs
 - uv, a fast modern Python package installer and resolver
 - Installing/building applications from source code
 - Builds that utilize dependencies already installed on Expanse
 - Spack based approach for custom environments
- ❖ Check if applications are already installed via modules or Singularity.
- ❖ Build new Singularity containers.
- ❖ Python or R based applications are available in
1. modules
 2. conda
 3. Singularity

Thank you!

Appendix

Creating your own module File

- An example of a bzip2 module file in classic **tcl** format (`/home/ychen64/my_module/bzip2/1.0.8/gcc-10.2.0`):

```
#!/Module
proc ModulesHelp { } {
    puts stderr {    bzip2    }
}
module-whatism {Description: bzip2}
set root    /home/ychen64/packages/bzip2-1.0.8
conflict    bzip2
if { ![is-loaded    gcc/10.2.0] } {
    module load gcc/10.2.0
}

prepend-path    CPATH                $root/include
prepend-path    LD_LIBRARY_PATH      $root/lib
prepend-path    LIBRARY_PATH         $root/lib
prepend-path    MANPATH              $root/share/man
prepend-path    PATH                 $root/bin
prepend-path    PKG_CONFIG_PATH      $root/lib/pkgconfig

[ychen64@exp-1-02 1.0.8]$ module load bzip2/1.0.8/gcc-10.2.0
```

Modules: create own module file

- Details about how to create own module file is placed in [Appendix](#)

Creating your own module File

- An example of a bzip2 module file in classic **tcl** format (`/home/ychen64/my_module/bzip2/1.0.8/gcc-10.2.0`):

```
#!/Module
proc ModulesHelp { } {
    puts stderr {    bzip2    }
}
module-whatism {Description: bzip2}
set root    /home/ychen64/packages/bzip2-1.0.8
conflict    bzip2
if { ![is-loaded    gcc/10.2.0] } {
    module load gcc/10.2.0
}

prepend-path    CPATH                $root/include
prepend-path    LD_LIBRARY_PATH      $root/lib
prepend-path    LIBRARY_PATH         $root/lib
prepend-path    MANPATH              $root/share/man
prepend-path    PATH                 $root/bin
prepend-path    PKG_CONFIG_PATH      $root/lib/pkgconfig
```



[ychen64@exp-1-02 1.0.8]\$ module load bzip2/1.0.8/gcc-10.2.0

SDSC

UCSan Diego 61

SDSC

UCSan Diego 14

Creating your own module File

- An example of an R module file in **Lmod** format

([/home/ychen64/my_module/r/4.5.3.lua](#)):

```
-- -*- lua -*-
whatis([[Name : r]])
whatis([[Version : 4.5.3]])
whatis([[Short description : R is 'GNU S', a freely available language and environment for statistical
computing. Please consult the R project homepage for further information.]])

help([[R is 'GNU S', a freely available language and environment for statistical. Please consult the R project
homepage for further information.]])

if not isloaded("gcc/10.2.0") then
    load("gcc/10.2.0")
end
if not isloaded("ncurses/6.2") then
    load("ncurses/6.2")
end
if not isloaded("intel-mkl/2019.1.144") then
    load("intel-mkl/2019.1.144")
end
prepend_path("PATH", "/home/ychen64/R-4.5.3/bin", ":")
setenv("RHOME", "/home/ychen64/R-4.5.3")
```

Creating your own module File

- Add the path to the key to the MODULEPATH environment variable:

```
export MODULEPATH=~/.my_module:$MODULEPATH
```

- Then try loading:

```
module purge
```

```
module load cpu slurm sdsc
```

```
module load bzip2/1.0.8/gcc-10.2.0
```

Why is Singularity so popular on HPC?

- On a typical supercomputer cluster like Expanse

You can't

- run `yum` or `apt-get` to facilitate the installation of a complex software
- update system libraries such as `glibc` for your applications
- easily share workflow with colleagues to let them reproduce your test results

Why is Singularity widely used on HPC?

> For more details on why Singularity, go to [Appendix](#)

Why is Singularity so popular on HPC?

- On a typical supercomputer cluster like Expanse
 - You can't
 - run `yum` or `apt-get` to facilitate the installation of a complex software
 - update system libraries such as `glibc` for your applications
 - easily share workflow with colleagues to let them reproduce your test results

SDSC UC San Diego

Why use Singularity

- Supercomputer cluster user may need a tool that can:
 - make software installation much easier
 - when yum or apt-get is available
 - can freely update any system libraries for the software
 - the software is already built and sharable in some free repositories
 - offer preferred Linux distribution
 - DIY, possibly no need to ask HPC staff for help
 - easily share workflow with colleagues (so they can reproduce the tests)
 - run software without any performance loss on the cluster
- Singularity is an excellent tool to achieve the goals above!

Why use Singularity ... more

- Not only a secure tool for containerizing software on HPC, but Singularity can also:
 - natively support
 - HPC interconnects such as InfiniBand
 - Nvidia GPU cards
 - MPI
 - easily be transported and shared as **single container image**
 - help to reproduce results on different clusters
 - integrate with popular workload managers such as Slurm

Summary: Why use Singularity

You can't	Now with Singularity you can
• run <code>yum</code> or <code>apt-get</code> to facilitate the installation of a complex software	• <code>yum install</code> / <code>apt-get install</code>, or even directly get an image from the developers
• update system libraries such as <code>glibc</code> for your applications	• build an image with a newer and preferred Linux distribution
• easily share workflow with colleagues to let them reproduce your test results	• share an image

R applications/packages via modules

- Several R packages are available via modules
- Use `module spider r-` to find all installed apps/libraries.
- Examples
 - `r-biobase`, `r-deseq2`, `r-doparallel`, `r-ggplot2`
- Use modules-based approach if all the packages needed are in modules **but pay attention to the package version available via modules.**
- If further installs are needed, it is best to install from source, Miniforge3 conda/mamba install or Singularity approach.
- Don't mix the R apps/libraries from the modules with ones from source, conda installs or a container. They will likely conflict/fail.



Install R packages from source

➤ Building R package from source

- R packages can be installed with `install.packages()`

Example of installing R packages with `install.packages()`



R console on HPC

```
> install.packages("praise")
* installing *source* package 'praise' ...
g++ -std=gnu++17 -I"/home/ychen64/R-4.5.3/lib64/R/include" -DNDEBUG -Iinclude -I../inst/include -DDUCKDB_DISABLE_PRINT -
```



R console on Windows PC

```
> install.packages("praise")
* installing *binary* package 'praise' ...
```

- R packages installed with Devtools and Bioconductor are generally from source
- RStudio in Singularity container may build R packages in binary (configuration required)
- R packages installed by Conda are pre-built binaries

Install R packages via conda install

```
(base) [ychen64@exp-4-57 ~]$ conda search r-base==4.5*
```

```
Loading channels: done
```

# Name	Version	Build	Channel
r-base	4.5.1	h14df4e6_3	conda-forge
r-base	4.5.1	h85845a0_0	conda-forge
r-base	4.5.1	h85845a0_1	conda-forge
r-base	4.5.1	h85845a0_2	conda-forge
r-base	4.5.2	h14df4e6_0	conda-forge
r-base	4.5.2	h1fbe982_4	conda-forge
r-base	4.5.2	h835929b_2	conda-forge
r-base	4.5.2	h835929b_3	conda-forge
r-base	4.5.2	hc038350_1	conda-forge
r-base	4.5.3	h15dba0b_1	conda-forge
r-base	4.5.3	h1a7235e_0	conda-forge

➤ Pros of R packages via conda install

- fast install small package(s) from existing external repo
- can be properly isolated within conda environment

➤ Cons

- packages may not be the same as official ones from CRAN
- dependencies are hard to change or even locked
- may mix conda channels of conda-forge, bioconda and r

Example of building R packages via Singularity

```
[ychen64@exp-9-55 ~]$ export SINGULARITY_TMPDIR=/scratch/$USER/job_${SLURM_JOBID} # avoid using /tmp
[ychen64@exp-9-55 ~]$ module load singularitypro
[ychen64@exp-9-55 ~]$ singularity build excerpt.sif docker://rkitchen/excerpt
WARNING: 'nodev' mount option set on /scratch, it could be a source of failure during build process
INFO: Starting build...
Getting image source signatures
Copying blob a59ce8f0c359 done |
Copying blob 5e35d10a3eba done |
Copying blob 236bb8549592 done |
Copying blob cc17f052e960 done |
Copying blob be036d7c9474 done |
Copying blob 6059ce0d04ff done |
Copying blob e6adcfd80d7c done |
```

Example of using R via Singularity

```
#!/bin/bash
```

```
#SBATCH --job-name="excerpt-test"
```

```
#SBATCH --output="excerpt.%j.%N.out"
```

```
#SBATCH --partition=shared
```

```
#SBATCH --nodes=1
```

```
#SBATCH --ntasks-per-node=1
```

```
#SBATCH --cpus-per-task=8
```

```
#SBATCH --mem=16G
```

```
#SBATCH --account=XYZ123
```

```
#SBATCH -t 04:00:00
```

```
### Modules
```

```
module reset
```

```
module load singularitypro
```

```
### Run the job
```

```
singularity run --bind $SLURM_SUBMIT_DIR/input:/exceRptInput --bind $SLURM_SUBMIT_DIR/output:/exceRptOutput --bind  
/expansE/projects/qstore/data/excerpt/hg38:/exceRpt_DB/hg38 /expansE/projects/qstore/installs/containers/singularity/excerpt /excerpt.sif  
INPUT_FILE_PATH=/exceRptInput/SRR026761.sra
```


More R for HPC training at SDSC

➤ SDSC Webinar: R for HPC

- R package management and user environment configuration on the HPC cluster
- R parallelism in HPC, such as using the "parallel" package in R and a few related packages to parallelize and enhance the performance of R programs.

<https://education.sdsc.edu/training/interactive/index.html?id=202512-R-for-HPC>